

Design: Serving Fine-Tuned Models to End Users v3

1. Problem Statement

After a fine-tuning task completes, the LoRA adapter is encrypted and uploaded to OG Storage. Currently, there is no way for end users to **use** these fine-tuned models for inference. Users must download, decrypt, and self-host the adapter — a poor experience that blocks adoption.

Goal: Allow users to send inference requests to their fine-tuned LoRA models through the existing OG Compute Network, with proper authentication and billing, without requiring users to run their own infrastructure.

1.1 Requirements

1. **Shared base model + multiple LoRA adapters:** A single GPU hosts one base model with many user-specific LoRA adapters attached dynamically.
2. **Reuse inference module's contract, auth, and billing:** Fine-tuned model serving MUST use the existing `InferenceServing` contract, session token authentication, per-token billing, and TEE-based settlement — not a parallel system.
3. **Hardware isolation:** Inference GPU(s) must be separate from training GPU(s) to avoid resource contention.
4. **Multi-tier LoRA caching:** Not all adapters fit in GPU memory simultaneously. Inactive adapters should be offloaded (GPU → CPU → Disk → OG Storage) and restored on demand.
5. **Access control:** Only the user who created a fine-tuning task (or users they authorize) can query their adapter.

2. Current Architecture Overview

2.1 Fine-Tuning Module (`api/fine-tuning/`)

Task lifecycle:

User creates task → Setup (download data/model) → Training (Docker/LoRA) → Finalize (encrypt + upload to OG Storage) → Deliver → User acknowledges → Settlement

Key artifacts after training:

- LoRA adapter directory on disk (`output_model/`)
- Encrypted LoRA uploaded to OG Storage (`OutputRootHash`)
- `EncryptedSecret` (AES key encrypted with user's public key)

Contract: `FineTuningServing` — methods: `addOrUpdateService` , `addDeliverable` , `acknowledgeDeliverable` , `settleFees` .

2.2 Inference Module (`api/inference/`)

Request flow:

User → POST `/v1/proxy/chat/completions`

- Session token validation (`app-sk-<base64(token|sig)>`)
- Balance check (`inputFee + unsettledFee + reservation`)
- Create DB Request record
- Forward to provider's target URL
- Extract usage (`prompt_tokens, completion_tokens`)
- Compute fee (`inputPrice × inputTokens + outputPrice × outputTokens`)
- Update DB Request with fee
- [Later] TEE signs batch → `settleFees` on-chain

Key mechanisms:

- **Contract registration:** `InferenceServing.addOrUpdateService(ServiceParams{URL, model, inputPrice, outputPrice, ...})`
- **Auth:** Session token with expiry, nonce, on-chain revocation check, address recovery
- **Billing:** Per-token pricing, DB Request records, unsettled fee tracking
- **Settlement:** TEE signs EIP-712 digest of request batch → `settleFees` on-chain

2.3 Gap Analysis (Why PR #374 Was Insufficient)

Capability	Inference Module	PR #374 (fine-tuning/serving)
Contract registration	<code>addOrUpdateService()</code>	No-op stub
Auth	Session token + on-chain verification	Static message signature, replayable
Billing	Per-token, DB records, unsettled tracking	None
Settlement	TEE-signed batch → on-chain	None
Service discovery	On-chain, public	Not discoverable

Conclusion: PR #374 built an isolated inference server without connecting to the economic model. The correct approach is to register fine-tuned models as inference services and route traffic through the inference proxy.

3. Proposed Architecture

3.1 Current Inference Broker Limitation

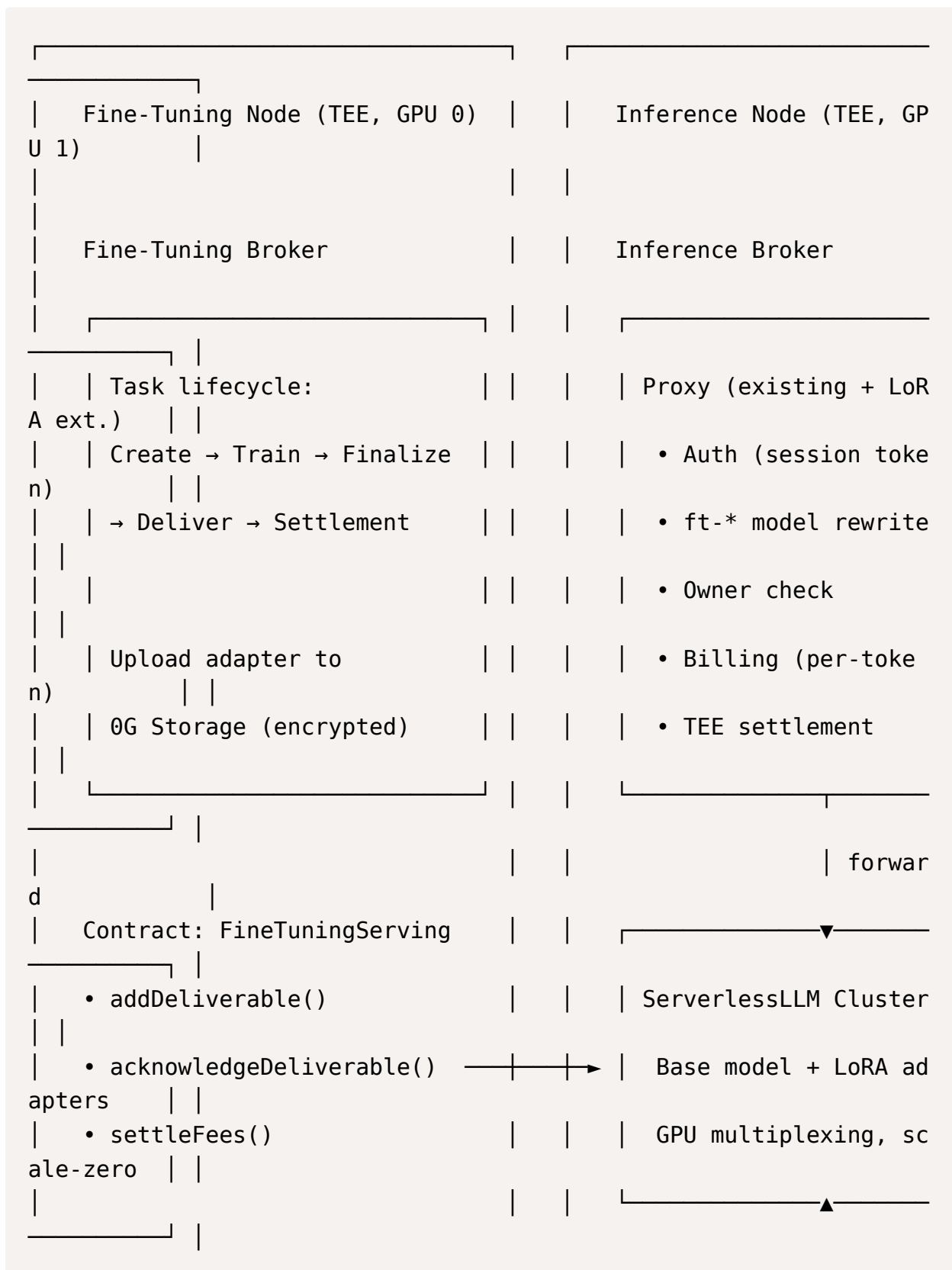
Critical constraint: The current inference broker only supports **one service per instance**.

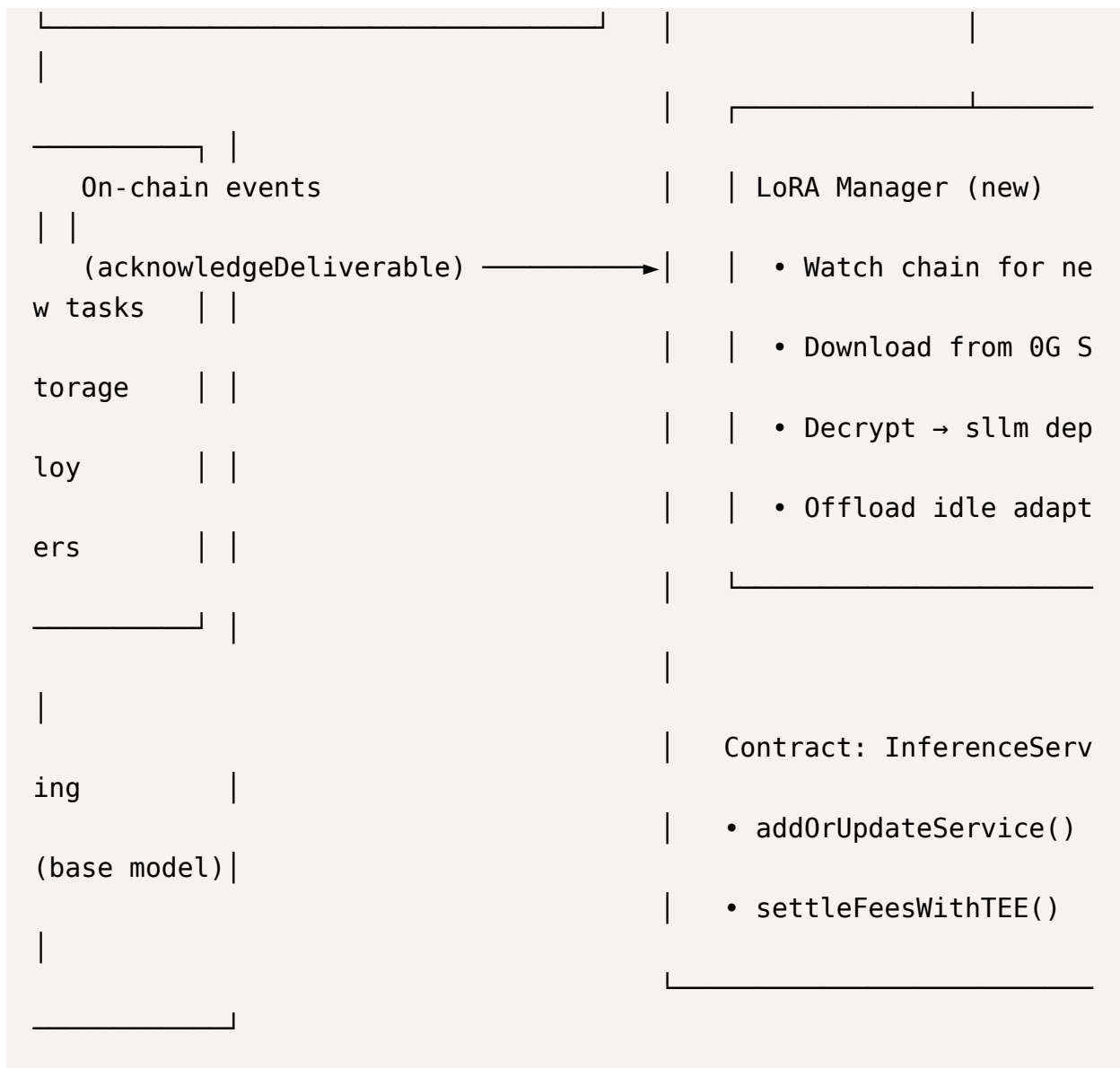
```
// config/config.go – singular Service, not Services[]
type Config struct {
    Service Service `yaml:"service"` // ONE service only
    ...
}

// proxy/proxy.go – singular target
type Proxy struct {
    serviceTarget string // ONE target URL
    serviceType   string // ONE service type
    ...
}
```

Each inference broker instance = one vLLM backend = one model. To serve fine-tuned LoRA adapters (which are multiple models on a shared base), the inference broker **must be extended to support multiple services**.

3.2 High-Level Flow





Key: The two modules run on **separate TEE nodes** with separate databases. They communicate via **on-chain events** (not shared DB). The fine-tuning contract emits events when tasks are delivered/acknowledged; the inference module's LoRA Manager watches these events to discover new adapters.

3.3 Contract Constraint: One Provider = One Service

The `InferenceServ` contract uses `keccak256(provider_address)` as the service key:

```
function _key(address provider) internal pure returns (bytes32) {
    return keccak256(abi.encode(provider));
}
```

```
}
```

This means **one wallet address can only register one service**. We cannot register each LoRA adapter as a separate on-chain service.

Solution: Register the **base model** as the single on-chain service. All LoRA adapters share the same `inputPrice` and `outputPrice`. The broker handles LoRA routing internally — no contract changes required.

Unified pricing rationale: During inference, the compute cost for different LoRA adapters is essentially the same (same base model, same GPU, same token generation). The difference is in training cost (different dataset sizes, LoRA ranks), which is already billed separately via the `FineTuningServing` contract.

3.4 Key Design Decisions

Decision 1: Inference module manages vLLM/ServerlessLLM and LoRA; fine-tuning module only trains.

The inference module already owns the vLLM process and the proxy. It should also manage LoRA adapters, since adapters are part of the serving pipeline. The fine-tuning module produces LoRA adapters and notifies inference when a new one is ready.

Decision 2: One on-chain service, LoRA routing inside broker.

The broker registers one service with the base model name (e.g., `Qwen2.5-7B`). All LoRA adapters share the same `inputPrice` and `outputPrice`. The broker internally maps the `model` field in user requests to the correct LoRA adapter and forwards to ServerlessLLM with the right `lora_adapter_name`.

```
On-chain: ONE service registered
  model: "Qwen2.5-7B"
  inputPrice: X, outputPrice: Y
  url: "<http://broker:3080>"
```

```
User request: { "model": "ft-Qwen2-5-7B-abc123", ... }
```

```
  |
  | Broker maps internally
  |
```



```
Forward to ServerlessLLM: { "model": "Qwen2.5-7B",
```

```
"lora_adapter_name": "ft-Qwen2-5-7B-a
bc123", ... }
```

Decision 3: Access control via owner check in broker.

For requests with `model` starting with `ft-`, the broker checks that the requester's address matches the fine-tuning task owner. This is a lightweight DB lookup before forwarding.

Decision 4: Billing uses the registered service's per-token pricing.

All LoRA adapters bill at the same rate as the base model. The existing billing pipeline (token counting → fee computation → TEE settlement) works unchanged.

3.5 Component Responsibilities

Component	Where	Responsibility
Fine-Tuning Worker	fine-tuning node (TEE)	Train LoRA adapter, encrypt + upload to OG Storage, <code>addDeliverable()</code> on-chain
LoRA Manager	inference node (new component)	Watch chain events, download/decrypt adapters from OG Storage, manage adapters in ServerlessLLM (<code>sllm deploy</code> / <code>sllm delete</code>), offload idle adapters
Proxy	inference node (minimal changes)	Map <code>ft-*</code> model names to <code>lora_adapter_name</code> , owner check, forward to ServerlessLLM
ServerlessLLM	inference node (new dependency)	Base model + LoRA inference, GPU multiplexing, scale-to-zero
FineTuningServing contract	on-chain	Task lifecycle, deliverable tracking, training fee settlement
InferenceServing contract	on-chain	Service registration (base model), inference fee settlement

3.6 Required Inference Module Changes

Area	Current	Required Change
Proxy	Forwards to single vLLM	Add model name mapping: <code>ft-*</code> → rewrite request body with <code>lora_adapter_name</code>
Config	Single <code>targetUrl</code> to vLLM	Point to ServerlessLLM endpoint instead; add <code>lora</code> config section
Owner check	None	New middleware: for <code>ft-*</code> models, verify requester == task owner (from LoRA Manager's in-memory map)

Area	Current	Required Change
LoRA Manager	None	New component: watch FineTuningServing contract events, download/decrypt adapters from OG Storage, manage ServerlessLLM lifecycle
Local DB	Request records only	Add <code>lora_adapters</code> table for crash recovery (adapter metadata)
Contract	Registers one service	No change — still one service, base model name
Billing	Per-token with one price	No change — all LoRA adapters use same price
Settlement	TEE batch per provider	No change
vLLM management	Broker manages vLLM directly	Replace with ServerlessLLM (managed separately via Docker Compose)

3.7 Inter-Module Communication

Constraint: Fine-tuning and inference run on **separate TEE nodes** with separate databases. They cannot share a MySQL instance. Therefore, inter-module communication uses **on-chain events**.

The flow:

1. Fine-tuning task completes → fine-tuning broker calls `FineTuningServing.addDeliverable()` on-chain
2. User acknowledges result → calls `acknowledgeDeliverable()` on-chain
3. Inference module's LoRA Manager watches the `FineTuningServing` contract for `DeliverableAcknowledged` events
4. When detected: extract task info (adapter OG Storage root hash, user address) → download from OG Storage → decrypt → deploy to ServerlessLLM

Why trigger on acknowledge (not deliver)? The user must explicitly confirm they want the fine-tuned result before the provider starts serving it. If the user never acknowledges (e.g., unhappy with quality), the adapter is NOT deployed for inference — no wasted GPU resources.

Adapter file transfer: The inference node downloads the encrypted adapter from **OG Storage** (not from the fine-tuning node's disk). This avoids any direct file sharing between TEE nodes. The provider holds the decryption key (or re-derives it) to decrypt the adapter locally.

4. Detailed Design

4.1 Config Changes (Minimal)

The broker config only needs a small addition — a `lora` section for the LoRA Manager:

```
# Existing service config – unchanged
service:
  targetUrl: "<http://sllm:8343>"      # point to ServerlessLLM instead of vLLM
  type: "chatbot"
  model: "Qwen2.5-7B"                # base model name (registered on-chain)
  inputPrice: "10000000"
  outputPrice: "10000000"

# New: LoRA serving config
lora:
  enable: true
  baseModel: "Qwen2.5-7B"
  loraModulesDir: "/data/lora-modules"
  offloadAfterMinutes: 60
  enableColdStorage: true
  fineTuningContractAddress: "0x1234..." # FineTuningServing contract to watch events
  chainRpcUrl: "<https://evmrpc-testnet.0g.ai>"
  pollBlockIntervalSeconds: 5
```

Key changes from previous version:

- `fineTuningDbProvider` removed — no shared DB between TEE nodes
- `fineTuningContractAddress` + `chainRpcUrl` added — LoRA Manager watches on-chain events instead
- `service.targetUrl` now points to **ServerlessLLM** (port 8343) instead of vLLM (port 8000)

Everything else stays the same — same model name on-chain, same prices, same contract registration.

4.2 LoRA Manager (New Component in Inference Module)

Lives in the inference module. Manages the LoRA adapter lifecycle via ServerlessLLM CLI + on-chain event watching.

Responsibilities:

1. Watch the `FineTuningServing` contract for `DeliverableAcknowledged` events (new completed + acknowledged tasks)
2. Download encrypted adapter from OG Storage → decrypt with provider key → place in `loraModulesDir`
3. Call `sllm deploy` to register adapter with ServerlessLLM
4. Maintain an in-memory map of `model name → adapter info (owner address, state, last access, OG root hash)`
5. Persist adapter metadata to local DB (inference node's own MySQL) for crash recovery
6. Track access times, offload idle adapters (`sllm delete`)
7. Restore from OG Storage on demand (download → decrypt → `sllm deploy`)

Startup sequence:

1. ServerlessLLM cluster is already running (Docker Compose)
2. Load adapter metadata from local DB (inference node's own MySQL)
3. For each known adapter: check if files exist on disk → `sllm deploy`
4. For missing files: re-download from OG Storage → decrypt → `sllm deploy`
5. Start event watcher (subscribe to `FineTuningServing` contract events)
6. Start offload loop (check idle adapters every N minutes)

Event watching detail:

```
// LoRA Manager subscribes to FineTuningServing contract events
func (m *LoRAManager) watchNewAdapters(ctx context.Context) {
    // Watch for DeliverableAcknowledged(taskID, userAddress, root
    Hash)
    // On event:
```

```

// 1. Download encrypted adapter from 0G Storage by rootHash
// 2. Decrypt using provider's key
// 3. Place in loraModulesDir/ft-<base>-<taskID>/
// 4. sllm deploy --lora-adapters "ft-<base>-<taskID>=<path>
>"
// 5. Add to in-memory map + persist to local DB
}

```

4.3 Proxy: Model Name Mapping

The only proxy change: intercept requests with `ft-*` model names, rewrite the body for ServerlessLLM.

```

func (p *Proxy) rewriteLoRARequest(reqBody []byte) ([]byte, string, error) {
    var body map[string]interface{}
    json.Unmarshal(reqBody, &body)

    modelName, _ := body["model"].(string)
    if !strings.HasPrefix(modelName, "ft-") {
        return reqBody, modelName, nil // base model: no rewrite
    }

    // Rewrite for ServerlessLLM:
    //   "model": "ft-Qwen2-5-7B-abc123"
    // becomes:
    //   "model": "Qwen2.5-7B",
    //   "lora_adapter_name": "ft-Qwen2-5-7B-abc123"
    body["model"] = p.ctrl.Config.Service.ModelType // base model
    name
    body["lora_adapter_name"] = modelName

    modified, _ := json.Marshal(body)
    return modified, modelName, nil
}

```

4.4 Complete User Journey (End to End)

The full lifecycle from account creation to inference settlement:

Phase A: Setup (user ↔ FineTuningServing contract)

- A1. User creates account with fine-tuning provider
 - `FineTuningServing.addAccount(providerAddress)`
 - Deposits funds (for training fee)
- A2. User creates fine-tuning task
 - `POST /fine-tuning/tasks` (to fine-tuning broker)
 - Uploads dataset to 0G Storage
 - Task enters queue: Pending → Setup → Training → Finalize → Delivered
- A3. Fine-tuning broker delivers result
 - Encrypted LoRA adapter uploaded to 0G Storage
 - `FineTuningServing.addDeliverable(taskID, rootHash, encrypted Secret)`
- A4. User acknowledges the result
 - `FineTuningServing.acknowledgeDeliverable(taskID)`
 - Training fee settled via `FineTuningServing.settleFees()`

Edge cases:

- User CANCELS task before completion → task stopped, no adapter, no inference
- User NEVER acknowledges → adapter stays in 0G Storage but is NOT deployed
 - for inference (LoRA Manager only triggers on acknowledge event)
- User acknowledges but adapter is corrupted → download fails, not deployed

Phase B: Inference Setup (user ↔ InferenceServing contract)

- B1. [Automatic] LoRA Manager on inference node detects Deliverable Acknowledged event
 - Downloads encrypted adapter from 0G Storage
 - Decrypts with provider's key

- Calls `sllm deploy` to load adapter into ServerlessLLM
- Adapter is now servable as "ft-<base>-<taskID>"

B2. User creates account with inference provider (SAME provider, but InferenceServing contract)

- InferenceServing: user calls addAccount(providerAddress)
- User deposits funds (for inference fees)
- (Note: this is the SAME provider address, but a DIFFERENT contract.)

The provider runs both fine-tuning and inference nodes.)

B3. User creates session token (done on USER's side, via SDK/CLI)

- User signs a message with their private key:
{ providerAddress, nonce, expiry }
- Encodes as: app-sk-<base64(message|signature)>
- This token is sent as Authorization header in all inference requests

(Where: user's local machine, using OG inference SDK. NOT on the broker.)

Phase C: Inference Request (user ↔ inference broker ↔ ServerlessLLM)

C1. User sends inference request:

POST /v1/proxy/chat/completions

Authorization: Bearer app-sk-<token>

Body: { "model": "ft-Qwen2-5-7B-abc123", "messages": [...] }

C2. Inference broker proxy:

- Validate session token → recover userAddress from signature
- Detect "ft-*" model → owner check:

LoRA Manager lookup: is userAddress the task owner? If not → 403

- Rewrite request body for ServerlessLLM:

{ "model": "Qwen2.5-7B", "lora_adapter_name": "ft-Qwen2-5-7B-abc123", ... }

- d. Check $\text{balance} \geq \text{estimatedFee} + \text{unsettledFees}$
(uses InferenceServing contract: `account(user, provider).balance()`)
 - e. Create Request record in broker's local DB
 - f. Forward to `<http://sllm:8343/v1/chat/completions>`
 - g. ServerlessLLM routes to correct LoRA adapter, returns response
 - h. Extract usage: `prompt_tokens`, `completion_tokens`
 - i. Compute $\text{fee} = \text{inputPrice} \times \text{promptTokens} + \text{outputPrice} \times \text{completionTokens}$
 - j. Update Request record with fee
- C3. Settlement (periodic, by inference broker's event service):
- a. Batch unprocessed Request records
 - b. TEE signs EIP-712 digest of the batch
 - c. Call `InferenceServing.settleFeesWithTEE(settlements[])`
 - d. Contract deducts from user's account balance → transfers to provider

Phase D: Adapter Lifecycle

- D1. Adapter goes idle (no requests for `offloadAfterMinutes`)
→ LoRA Manager calls ``sllm delete`` to unload from ServerlessLLM
→ Adapter files may remain on disk or be cleaned up
→ Inference requests for this adapter now return 503
- D2. User sends request for offloaded adapter
→ LoRA Manager detects adapter not loaded
→ Downloads from OG Storage → decrypts → ``sllm deploy``
→ Returns 503 with `Retry-After` (or blocks if `wait_for_model=true`)
- D3. Provider stops serving (removes inference service)
→ `InferenceServing.removeService()` → stake returned
→ All adapters become inaccessible

4.5 Access Control

Where: Access control for fine-tuned models is added to the **existing inference broker** (not a new broker). Specifically, it is a new middleware step in the existing proxy handler chain (`api/inference/internal/proxy/`).

Why not a separate broker? The existing inference broker already handles authentication (session tokens), billing (per-token), and settlement (TEE). Building a separate "fine-tune inference broker" would duplicate all of this. Instead, we extend the existing proxy with one additional check: for `ft-*` model names, verify the requester is the task owner.

Data source: The LoRA Manager maintains an in-memory map of adapter → owner, populated from on-chain events (DeliverableAcknowledged events contain the user address and task ID). No shared DB needed.

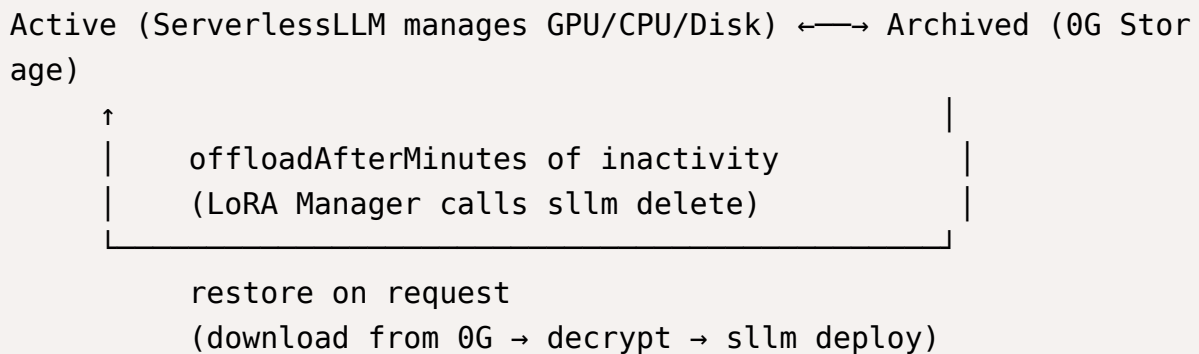
```
func (p *Proxy) checkLoRAOwnership(modelName, userAddress string)
error {
    if !strings.HasPrefix(modelName, "ft-") {
        return nil // base model: no owner check, standard access
    }
    adapter := p.loraManager.GetAdapter(modelName)
    if adapter == nil {
        return errors.New("model not found: " + modelName)
    }
    if !strings.EqualFold(adapter.OwnerAddress, userAddress) {
        return errors.New("access denied: you are not the owner of
this fine-tuned model")
    }
    return nil
}
```

Integration point: This check runs AFTER session token validation (which recovers `userAddress`) and BEFORE the request is forwarded to ServerlessLLM. The middleware chain in the proxy becomes:

```
Session token validation → Balance check → [NEW: LoRA owner check]
→ Request rewrite → Forward
```

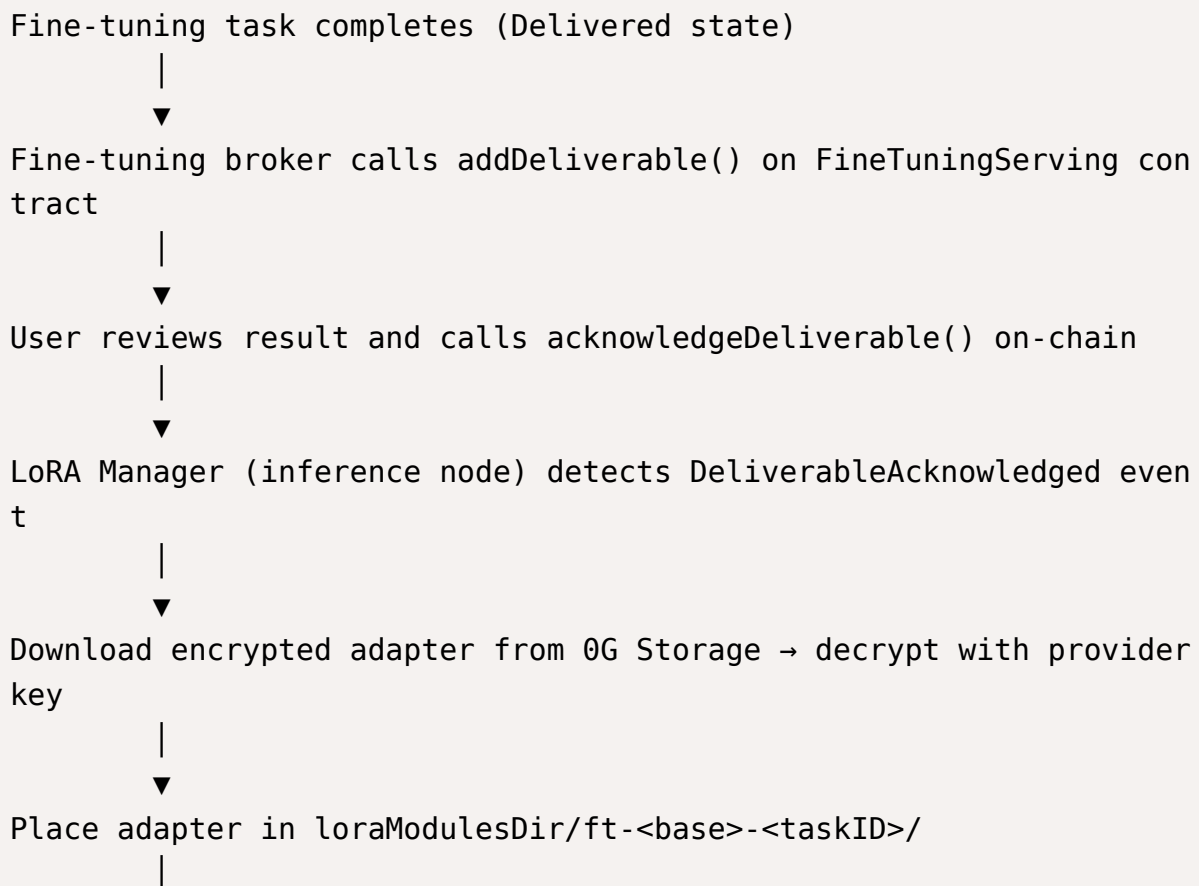
4.6 Multi-Tier LoRA Caching

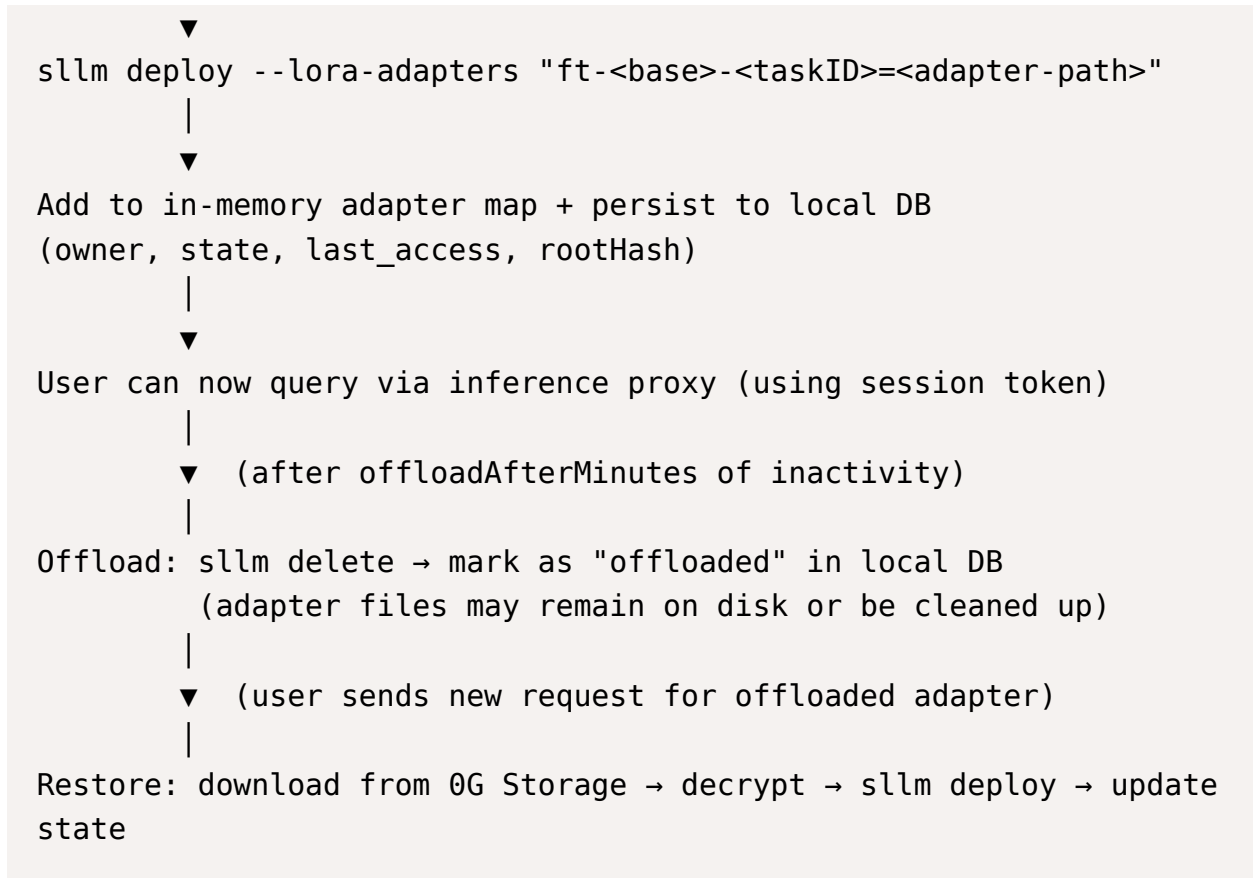
Managed by the LoRA Manager, using ServerlessLLM's built-in caching + our 0G Storage cold tier:



ServerlessLLM handles the fast tier (GPU ↔ CPU ↔ Disk) automatically. The LoRA Manager only manages the cold tier (Disk ↔ 0G Storage) for adapters that need to be fully offloaded.

4.7 Serving Lifecycle





5. ServerlessLLM Analysis

5.1 What ServerlessLLM Offers

[ServerlessLLM](#) (OSDI'24) is an open-source system for multi-model GPU multiplexing with 659 stars, Apache 2.0 license:

Feature	Description
Fast checkpoint loading	Custom binary format + O_DIRECT I/O → 6-10x faster than SafeTensors
GPU multiplexing	Run 10+ models on 1 GPU with fast switching and scale-to-zero
LoRA fine-tuning	PEFT LoRA backend (<code>ft_backend</code>) with job management, priority queue, timeout
LoRA serving	<code>sllm deploy --enable-lora --lora-adapters</code> → OpenAI-compatible inference
Storage-aware scheduling	Locality-driven server allocation minimizes loading latency
Live migration	Move running models between nodes with zero downtime

Feature	Description
OpenAI-compatible API	Drop-in replacement for <code>/v1/chat/completions</code> , <code>/v1/embeddings</code>
Docker/K8s ready	Docker Compose cluster (head + workers), Ray-based

5.2 Coverage Matrix

Requirement	ServerlessLLM	OG Must Build	Notes
LoRA fine-tuning	✓	—	PEFT backend, HuggingFace datasets, configurable LoRA rank/alpha
LoRA serving (base + adapters)	✓	—	vLLM/Transformers backend, named adapters
Multi-tier caching (GPU → CPU → Disk)	✓	—	Built-in, more mature than custom Go
Fast model loading (6-10x)	✓	—	<code>sllm-store</code> , O_DIRECT I/O, pinned memory DMA
Scale to zero	✓	—	Auto-scale per model, idle reclamation
GPU multiplexing	✓	—	Intelligent scheduling across models
On-chain contract registration	✗	✓	<code>InferenceServing.addOrUpdateService()</code>
Session token authentication	✗	✓	<code>app-sk-<base64(token\ sig)></code> , expiry, revocation
Per-token billing	✗	✓	<code>inputPrice × tokens + outputPrice × tokens</code>
TEE-signed settlement	✗	✓	EIP-712 batch settlement
OG Storage (cold tier)	✗	✓	Encrypted LoRA upload/download
Encrypted model delivery	✗	✓	AES-GCM + ECIES
Per-user access control	✗	✓	Task owner verification

5.3 Integration Approach: ServerlessLLM as Serving Engine Only

Keep the existing fine-tuning pipeline (Docker-based training, OG Storage, TEE) unchanged. Replace direct vLLM management with ServerlessLLM **for serving only**. The inference broker handles all traffic (auth, billing, settlement).

ServerlessLLM is NOT used for fine-tuning because: (1) it's not designed for TEE environments, (2) its `ft_backend` doesn't support encrypted OG Storage datasets, and (3) patching OG-specific features into its Python codebase would create a maintenance burden.

How it works:

1. Fine-tuning task completes → encrypted LoRA uploaded to OG Storage → `addDeliverable()` on-chain
2. User acknowledges → `acknowledgeDeliverable()` on-chain
3. LoRA Manager (inference node) watches chain events → downloads from OG Storage → decrypts → `sllm deploy`
4. User sends request → inference proxy (auth, owner check, billing) → rewrite ft-* model → forward to ServerlessLLM → response → fee computation
5. On idle timeout: `sllm delete` the adapter (on-chain service registration unchanged — it's the base model)
6. On restore: download from OG Storage → decrypt → `sllm deploy`

5.4 Risks and Mitigations

Risk	Impact	Mitigation
Ray dependency (ServerlessLLM is Ray-based)	Deployment complexity increases	Containerize ServerlessLLM cluster; fine-tuning broker connects via HTTP API
ServerlessLLM upstream breaking changes	Adapter management API may change	Pin version, maintain thin Go client wrapper
TEE environment compatibility	ServerlessLLM workers may not run in TEE	Run ServerlessLLM outside TEE; only fine-tuning training runs in TEE
Cold restore latency	Downloading from OG Storage + decryption adds time	Use <code>sllm-store</code> for fast disk → GPU loading after download; pre-warm popular adapters
Single point of failure	ServerlessLLM cluster down = no serving	Health checks + auto-restart; fallback to direct vLLM if needed

6. Implementation Plan

Phase 1: ServerlessLLM Setup + Proxy LoRA Routing

Goal: Deploy ServerlessLLM, add model name mapping and owner check to broker proxy.

1. Deploy ServerlessLLM cluster

- Set up ServerlessLLM head + worker(s) via Docker Compose
- Deploy base model with `-enable-lora`
- Verify OpenAI-compatible API (`/v1/chat/completions`)
- Pin ServerlessLLM version

2. Point broker `targetUrl` to ServerlessLLM

- Change `service.targetUrl` from `http://vllm:8000` to `http://sllm:8343`
- Verify base model inference still works end-to-end

3. Add LoRA request rewriting in proxy

- Detect `ft-*` in request `model` field
- Rewrite: `model` → base model name, add `lora_adapter_name`
- Forward to ServerlessLLM

4. Add owner-check middleware

- For `ft-*` models, look up adapter owner from LoRA Manager's in-memory map
- Reject if requester != owner

Phase 2: LoRA Manager + On-Chain Event Watching

Goal: Automate adapter lifecycle — discover via chain events, deploy, offload, restore.

1. Build Go client wrapper for ServerlessLLM CLI

- `DeployAdapter(baseModel, adapterName, adapterPath)` → `sllm deploy`
- `DeleteAdapter(baseModel, adapterName)` → `sllm delete`
- `ListModels()` → `sllm status`
- Health check, error handling, retry

2. Build LoRA Manager in inference module

- Subscribe to `FineTuningServing` contract events (DeliverableAcknowledged)
- On event: download from OG Storage → decrypt → `sllm deploy` → update adapter map

- Persist adapter metadata to inference node's local DB for crash recovery
 - Offload loop: track last access, `sllm delete` idle adapters
 - Restore: download from OG Storage → decrypt → `sllm deploy`
3. **Local adapter metadata table** (inference node's own DB, NOT shared)
 - Fields: taskID, userAddress, baseModel, adapterName, storageRootHash, state (active/offloaded/loading), lastAccess
 - Used for crash recovery: on restart, re-deploy known adapters
 4. **End-to-end test**
 - Fine-tune task → delivered → user acknowledges → LoRA Manager detects event → adapter deployed → user queries → billing → settlement

Phase 3: Optimization

1. `sllm-store` for faster cold restore
 - Convert adapters to ServerlessLLM's optimized format
 - Benchmark loading speedup
2. **Streaming billing**
 - Token counting from SSE chunks
3. **Adapter sharing**
 - Allow task owners to grant access to other users

Phase 4: Production Hardening

1. **Monitoring:** Prometheus metrics for active adapters, loading latencies, cache hit rates
 2. **Rate limiting:** Per-user request limits
 3. **Quota management:** Max adapters per user
 4. **Fault tolerance:** Auto-restart ServerlessLLM, re-deploy adapters on recovery
-

7. Open Questions

1. **LoRA decryption key management** (critical, needs TEE team input)

The fine-tuning node encrypts the LoRA adapter with the user's public key before uploading to OG Storage. The inference node (a separate TEE) needs the decrypted version. How does the inference node obtain the decryption key? This likely requires a provider-level key scheme: the fine-tuning node re-encrypts with a provider key so any of the provider's TEE nodes can decrypt.

2. On-chain event reliability

The LoRA Manager watches `FineTuningServing` contract events. It must handle: missed events during downtime (replay from checkpoint block number), chain reorgs (sufficient confirmation depth), and event indexing at scale.

3. User account across two contracts

A user must deposit to BOTH `FineTuningServing` (training fees) and `InferenceServing` (inference fees). The SDK/CLI should guide users through both deposits in a single workflow to avoid confusion.

4. Future: per-adapter pricing

Currently all LoRA adapters share the base model's price. If per-adapter pricing is needed later, the `InferenceServing` contract must be upgraded to support multi-service per provider (`keccak256(provider, serviceName)` as key).

5. ServerlessLLM version pinning

ServerlessLLM is pre-1.0 and actively developed. Pin to a specific commit and maintain a thin Go client wrapper to isolate API changes.

6. GPU resource isolation

GPU 0 for training, GPU 1 for inference, enforced via `CUDA_VISIBLE_DEVICES`. Need to define behavior for providers with variable GPU counts.

7. Provider restart recovery

On restart, LoRA Manager re-loads adapter metadata from local DB → checks adapter files on disk → `sllm deploy` for existing → downloads from OG Storage for missing. Need to handle partial failures and race conditions with incoming requests.

8. References

- [ServerlessLLM \(OSDI'24\)](#) — Fast multi-model serving with GPU multiplexing
- [ServerlessLLM Paper](#) — Architecture details, checkpoint loading optimization
- [ServerlessLLM Documentation](#) — Deployment and API reference

- vLLM LoRA support — Dynamic LoRA loading (used by ServerlessLLM internally)
- OG Inference Module — Contract, auth, billing, settlement code
- OG Fine-Tuning Module — Task lifecycle, OG Storage, TEE
- PR #373 — Fine-tuning monitoring (merged)
- PR #374 — Previous serving implementation (on hold, to be superseded by this design)